Apache Spark Developer Associate

Independent certification cheat sheet + 7-day practice plan | PySpark-first | Updated October 2026

Study plan: 7 days x 3-4 hours

Day	Focus / deliverable
1	Architecture, partitions, jobs, stages; explain lazy evaluation
2	DataFrame creation, schemas, read/write, null handling
3	Columns, string/date/array functions and SQL
4	Joins, aggregations and window functions
5	Performance: shuffle, caching, partitioning, plans
6	Build and test a trade-processing mini-project
7	Timed exercises, review incorrect answers, mock exam

1. Architecture and execution

Term	Meaning / exam relevance
Driver	Coordinates the application, plans work and schedules tasks.
Executor	Runs tasks and caches partition data.
Partition	Unit of distributed parallelism; not the same as a directory partition.
Job / stage / task	An action triggers a job; stages separate shuffle boundaries; tasks process partitions.
Lazy evaluation	Transformations construct plans; actions trigger computation.
Narrow vs wide	Narrow dependencies can avoid shuffle; wide dependencies may redistribute data.
Catalyst / AQE	Optimization of query plans; AQE can adapt based on runtime statistics.

Know: **filter** and **select** are transformations; **count**, **show** and **collect** are actions. Avoid collecting large data to the driver.

2. Essential PySpark API

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
schema = StructType([StructField("trade_id", StringType()),
                     StructField("pnl", DoubleType())])
df = spark.read.schema(schema).option("header", True).csv("/path/trades.csv")
df.printSchema()
clean = df.filter(F.col("pnl").isNotNull())
result = clean.groupBy("trade_id").agg(F.sum("pnl").alias("total_pnl"))
```

```
result.orderBy(F.desc("total_pnl")).show(10)
```

API group	Must know
Read/write	read.csv, read.parquet, write.mode, partitionBy, save
Columns	select, withColumn, withColumnRenamed, cast, alias
Nulls	isNull, isNotNull, fillna, dropna, coalesce
Arrays/strings/dates	explode, split, regexp_extract, to_date, datediff
Duplicates	distinct vs dropDuplicates(subset)
SQL	createOrReplaceTempView and spark.sql

3. Joins and windows

```
joined = trades.join(books, on="book_id", how="left")
from pyspark.sql.window import Window
w = Window.partitionBy("book_id").orderBy(F.desc("pnl"))
ranked = joined.withColumn("rn", F.row_number().over(w))
top = ranked.filter(F.col("rn") == 1)
```

Join types: inner, left, right, full, left_semi, left_anti, cross. Semi/anti joins output only columns from the left side. Ranking: row_number gives unique numbers; rank leaves gaps after ties; dense_rank does not.

4. Performance cheat sheet

Concept	Decision rule
Shuffle	Expensive redistribution; inspect when grouping or joining.
Broadcast	Useful for sufficiently small join side; not always faster.
repartition(n)	Can increase/decrease partitions; typically shuffles.
coalesce(n)	Usually decreases partitions without full shuffle.
cache / persist	Useful when reused; consumes memory/storage; unpersist when finished.
explain(True)	Inspect physical plan and join strategies.
Skew	A few partitions much larger than others; AQE or salting may help.
UDF	Prefer native Spark functions when possible for optimization.

```
df.explain(True)
print(df.rdd.getNumPartitions())
small = F.broadcast(reference_df)
joined = fact_df.join(small, "key")
reused = df.cache()
reused.count()   # materialize cache
reused.unpersist()
```

5. Hands-on project: trade processing

Create a synthetic 100,000-row trade dataset with trade_id, book, desk, instrument, timestamp, quantity, price and pnl. Read with explicit schema, validate nulls, deduplicate by trade_id, join reference data, aggregate by book, rank trades, write Parquet and compare execution plans.

Acceptance checks: record counts reconcile, no duplicate trade IDs, null P&L policy is explicit, aggregates match a small hand-calculated sample, and output can be re-read.

Stretch: demonstrate a skewed join and measure runtime before/after broadcast or repartitioning.

6. Self-test (answers below)

1. Which operation triggers a job: select, filter, or count?

2. How does left_anti differ from left_semi?

3. Why can collect() be dangerous?

4. When might repartition(100) make a job slower?

5. Which rank function does not leave gaps after ties?

Answers: 1 count; 2 unmatched vs matched left rows; 3 driver memory pressure; 4 excessive shuffle/tiny partitions; 5 dense_rank.

Exam-day checklist

Be fluent with PySpark DataFrame syntax, SQL, joins, null semantics, schemas, functions, windows, I/O and execution. Check the latest official exam guide before booking: https://www.databricks.com/learn/certification/apache-spark-developer-associate